

ADA - Análisis y Diseño de Algoritmos, 2026-1

Tarea 4: Semanas 10 y 11

Para entregar el martes 28 de abril de 2026

Problemas conceptuales a las 23:59 por BrightSpace

Problemas prácticos a las 23:59 en la arena de programación

Tanto los ejercicios como los problemas deben ser resueltos, pero únicamente las soluciones de los problemas deben ser entregadas. La intención de los ejercicios es entrenarlo para que domine el material del curso; a pesar de que no debe entregar soluciones a los ejercicios, usted es responsable del material cubierto en ellos.

Instrucciones para la entrega

Para esta tarea y todas las tareas futuras, la entrega de soluciones es *individual*. Por favor escriba claramente su nombre, código de estudiante y sección en cada hoja impresa entregada o en cada archivo de código (a modo de comentario). Adicionalmente, agregue la información de fecha y nombres de compañeros con los que colaboró; igualmente cite cualquier fuente de información que utilizó.

¿Cómo describir un algoritmo?

En algunos ejercicios y problemas se pide “dar un algoritmo” para resolver un problema. Una solución debe tomar la forma de un pequeño ensayo (es decir, un par de párrafos). En particular, una solución debe resumir en un párrafo el problema y cuáles son los resultados de la solución. Además, se deben incluir párrafos con la siguiente información:

- una descripción del algoritmo en castellano y, si es útil, pseudo-código;
- por lo menos un diagrama o ejemplo que muestre cómo funciona el algoritmo;
- una demostración de la corrección del algoritmo; y
- un análisis de la complejidad temporal del algoritmo.

Recuerde que su objetivo es comunicar claramente un algoritmo. Las soluciones algorítmicas correctas y descritas *claramente* recibirán alta calificación; soluciones complejas, obtusas o mal presentadas recibirán baja calificación.

Problemas conceptuales

1. Ejercicio 2.1: *Subset Sum* (Erickson, página 93).

Problemas prácticos

Hay cinco problemas prácticos cuyos enunciados aparecen a partir de la siguiente página.

A - Boggle Blitz

Source file name: `boggle.py`

Time limit: 1 second

In the game of Boggle, you are presented with an $N \times N$ table of letters. The object is to find words in the mix of letters. A word may be started anywhere in the table and is constructed by forming a chain of adjacent letters. Adjacent means diagonal, vertical or horizontal. A word cannot use any character from the table more than once.

Here is an example of a 4×4 table:

```
bile
tglp
aest
here
```

The following is a partial list of legal words that can be found using the above rules:

```
bill
gates
slept
here
```

Input

The input begins with a single positive integer on a line by itself indicating the number of the cases following, each of them as described below. This line is followed by a blank line, and there is also a blank line between two consecutive inputs.

In “Boggle Blitz” you will be given an integer number, N , on a single line. N is the number of characters on each side of the table, giving a total of N^2 characters. N can range from 1 to 20. N lines of N characters each will follow giving you the arrangement of the table.

Your task is to find all the legal words in the table. **Be sure that your program is efficient!**

In case you forgot to bring a dictionary, you are in luck because we are not using English words. Instead, we have redefined “word” to mean an increasing (by ASCII value) chain of characters from length 3 to length N^2 . “ABCDE” is a legal five letter word. “MICROSOFT” is not legal because the sequence is not increasing. “BILL” is also illegal because L is not greater than —L—. “BIL”, however, is legal.

The input must be read from standard input.

Output

For each test case, the output must follow the description below. The outputs of two consecutive cases will be separated by a blank line.

Your program should output the list of unique words sorted according to the following criteria:

1. Shorter words are before longer words
2. Words are sorted lexicographically by ASCII value
3. Add no blank lines or spaces. If no legal words can be found, print nothing.

The output must be written to standard output.

Sample Input	Sample Output
2	dop dot
3	eno enp ent
one top dog	eop eot
4	gop got nop not enop enot
abcd bcda cdab dabc	abc abd acd bcd abcd

B - Garden of Eden

Source file name: `eden.py`

Time limit: 1 second

Cellular automata are mathematical idealizations of physical systems in which both space and time are discrete, and the physical quantities take on a finite set of discrete values. A cellular automaton consists of a lattice (or array), usually infinite, of discrete-valued variables. The state of such automaton is completely specified by the values of the variables at each place in the lattice. Cellular automata evolve in discrete time steps, with the value at each place (cell) being affected by the values of variables at sites in its neighborhood on the previous time step. For each automaton there is a set of rules that define its evolution.

For most cellular automata there are configurations (states) that are unreachable: no state will produce them by the application of the evolution rules. These states are called Gardens of Eden for they can only appear as initial states. As an example consider a trivial set of rules that evolve every cell into 0; for this automaton any state with non-zero cells is a Garden of Eden.

In general, finding the ancestor of a given state (or the non-existence of such ancestor) is a very hard, compute intensive, problem. For the sake of simplicity we will restrict the problem to 1-dimensional binary finite cellular automata. This is, the number of cells is a finite number, the cells are arranged in a linear fashion and their state will be either '0' or '1'. To further simplify the problem each cell state will depend only on its previous state and that of its immediate neighbors (the one to the left and the one to the right).

The actual arrangement of the cells will be along a circumference, so that the last cell is next to the first.

Given a circular binary cellular automaton you must find out whether a given state is a Garden of Eden or a reachable state. The cellular automaton will be described in terms of its evolution rules. For example, the table below shows the evolution rules for the automaton: $Cell = XOR(Left; Right)$.

Left [i - 1]	Cell [i]	Right [i + 1]	New State	
0	0	0	0	$0 * 2^0$
0	0	1	1	$1 * 2^1$
0	1	0	0	$0 * 2^2$
0	1	1	1	$1 * 2^3$
1	0	0	1	$1 * 2^4$
1	0	1	0	$0 * 2^5$
1	1	0	1	$1 * 2^6$
1	1	1	0	$0 * 2^7$
				90 = Automaton Identifier

Notice that, with the restrictions imposed to this problem, there are only 256 different automata. An identifier for each automaton can be generated by taking the New State vector and interpreting it as a binary number (as shown in the table). For instance, the automaton in the table has identifier 90. The *Identity* automaton (every state evolves to itself) has identifier 204.

Input

The input will consist of several test cases. Each input case will describe, in a single line, a cellular automaton and a state. The first item in the line will be the identifier of the cellular automaton you must work with. The second item in the line will be a positive integer N ($4 \leq N \leq 32$) indicating the number of cells for this test case. Finally, the third item in the line will be a state represented by a string of exactly N zeros and ones. Your program must keep reading lines until the end of the input (end of file).

The input must be read from standard input.

Output

If an input case describes a Garden of Eden you must output the string GARDEN OF EDEN. If the input does not describe a Garden of Eden (it is a reachable state) you must output the string REACHABLE.

The output for each test case must be in a different line.

The output must be written to standard output.

Sample Input	Sample Output
0 4 1111	GARDEN OF EDEN
204 5 10101	REACHABLE
255 6 000000	GARDEN OF EDEN
154 16 1000000000000000	GARDEN OF EDEN

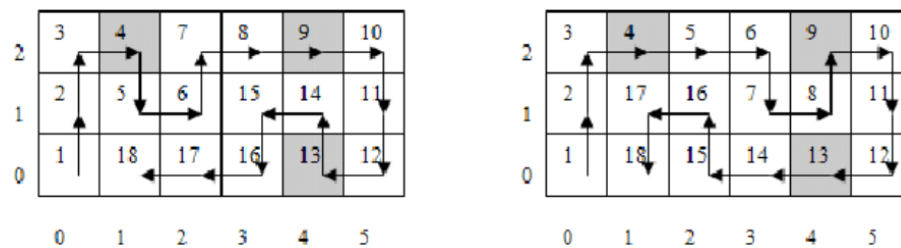
C - Robots on Ice

Source file name: `ice.py`

Time limit: 6 seconds

Inspired by the ice sculptures in Harbin, the members of the programming team from Arctic University of Robotics and Automata have decided to hold their own ice festival when they return home from the contest. They plan to harvest blocks of ice from a nearby lake when it freezes during the winter. To make it easier to monitor the thickness of the ice, they will lay out a rectangular grid over the surface of the lake and have a lightweight robot travel from square to square to measure ice thickness at each square in the grid. Three locations in the grid are specified as “check-in” points and the robot is supposed to radio a progress report from these points when it is one-fourth, one-half, and three-fourths of the way through its tour of inspection. To avoid unnecessary wear and tear on the surface of the ice, the robot must begin its tour of the grid from the lower left corner, designated in (row,column) coordinates as (0,0), visiting every other grid location exactly once and ending its tour in row 0, column 1. Moreover, if there are multiple tours that the robot can follow, then a different one is to be used each day. The robot is able to move only one square per time step in one of the four compass directions: north, south, east, or west.

You are to design a program that determines how many different tours are possible for a given grid size and a sequence of three check-in points. For example, suppose the lake surface is marked off in a 3×6 grid and that the check-in points, in order of visitation, are (2,1), (2,4), and (0,4). Then the robot must start at (0,0) and end at (0,1) after visiting all 18 squares. It must visit location (2,1) on step 4 ($= \lfloor 18/4 \rfloor$), location (2,4) on step 9 ($= \lfloor 18/2 \rfloor$), and location (0,4) on step 13 ($= \lfloor 3 \times 18/4 \rfloor$). There are just two ways to do this (see Figure below). Note that when the size of the grid is not divisible by 4, truncated division is used to determine the three check-in times.



Note that some configurations may not permit any valid tours at all. For example, in a 4×3 grid with check-in sequence (2,0), (3,2), and (0,2), there is no tour of the grid that begins at (0,0) and ends at (0,1).

Input

The input contains several test cases. Each test case begins with a line containing two integers m and n , where $2 \leq m, n \leq 8$, specifying the number of rows and columns, respectively, in the grid. This is followed by a line containing six integer values r_1, c_1, r_2, c_2 , and r_3, c_3 , where $0 \leq r_i < m$ and $0 \leq c_i < n$ for $i = 1, 2, 3$. Following the last test case is a line containing two zeros.

The input must be read from standard input.

Output

Display the case number, beginning at 1, followed by the number of possible tours that begin at row 0, column 0, end at row 0, column 1, and visit row r_i , column c_i at time $\lfloor i \cdot m \cdot n/4 \rfloor$ for $i = 1, 2, 3$. Follow the format of the sample output.

The output must be written to standard output.

Sample Input	Sample Output
3 6 2 1 2 4 0 4 4 3 2 0 3 2 0 2 0 0	Case 1: 2 Case 2: 0

D - Super Number

Source file name: `super.py`

Time limit: 1 second

Don't you think 162456723 very special? Look at the picture below if you are unable to find its speciality. ($a|b$ means ' b is divisible by a ').

Given n, m ($0 < n < m < 30$), you are to find a m -digit positive integer X such that for every i ($n \leq i \leq m$), the first i digits of X is a multiple of i . If more than one such X exists, you should output the lexicographically smallest one. Note that the first digit of X should not be 0.

Input

The first line of the input contains a single integer t ($t \geq 0$), the number of test cases followed. For each case, two integers n and m are separated by a single space.

The input must be read from standard input.

Output

For each test case, print the case number and X . If no such number, print '-1'.

The output must be written to standard output.

Sample Input	Sample Output
2 1 10 3 29	Case 1: 1020005640 Case 2: -1

E - Mapping the Swaps

Source file name: `swaps.py`

Time limit: 1 second

Sorting an array can be done by swapping certain pairs of adjacent entries in the array. This is the fundamental technique used in the well-known bubble sort. If we list the identities of the pairs to be swapped, in the sequence they are to be swapped, we obtain what might be called a swap map. For example, suppose we wish to sort the array A whose elements are 3, 2, and 1 in that order. If the subscripts for this array are 1, 2, and 3, sorting the array can be accomplished by swapping A_2 and A_3 , then swapping A_1 and A_2 , and finally swapping A_2 and A_3 . If a pair is identified in a swap map by indicating the subscript of the first element of the pair to be swapped, then this sorting process would be characterized with the swap map 2 1 2.

It is instructive to note that there may be many ways in which swapping of adjacent array entries can be used to sort an array. The previous array, containing 3 2 1, could also be sorted by swapping A_1 and A_2 , then swapping A_2 and A_3 , and finally swapping A_1 and A_2 again. The swap map that describes this sorting sequence is 1 2 1.

For a given array, how many different swap maps exist? A little thought will show that there are an infinite number of swap maps, since sequential swapping of an arbitrary pair of elements will not change the order of the elements. Thus the swap map 1 1 1 2 1 will also leave our array elements in ascending order. But how many swap maps of minimum size will place a given array in order? That is the question you are to answer in this problem.

Input

The input data will contain an arbitrary number of test cases, followed by a single '0'. Each test case will have a integer n that gives the size of an array, and will be followed by the n integer values in the array. In no test case will n be larger than 5.

The input must be read from standard input.

Output

For each test case, print a message similar to those shown in the sample output below.

The output must be written to standard output.

Sample Input	Sample Output
2 9 7 2 12 50 3 3 2 1 3 9 1 5 0	There are 1 swap maps for input data set 1. There are 0 swap maps for input data set 2. There are 2 swap maps for input data set 3. There are 1 swap maps for input data set 4.